

~~ai_services.py~~

ai_services.py

AI Whisper API (OpenAI Whisper API)

import os

OpenAI API key, (env_utils.py) (OpenAI Whisper API)

try:

from env_utils import OPENAI_API_KEY

if OPENAI_API_KEY:

from openai import OpenAI

connect timeout: 10s, read timeout: 300s (5min)

import httpx

openai_client = OpenAI(

api_key=OPENAI_API_KEY,

timeout=httpx.Timeout(10.0, read=300.0) # 10s, 5min

)

OPENAI_AVAILABLE = True

OPENAI_ERROR = None

else:

openai_client = None

OPENAI_AVAILABLE = False

OPENAI_ERROR = "OPENAI_API_KEY is not set or invalid."

except ImportError:

openai_client = None

OPENAI_AVAILABLE = False

OPENAI_ERROR = "openai package is not installed."

except Exception as e:

openai_client = None

OPENAI_AVAILABLE = False

OPENAI_ERROR = str(e)

STT using OpenAI Whisper API

STT_AVAILABLE = OPENAI_AVAILABLE

STT_ERROR = OPENAI_ERROR

def transcribe_audio(file_path, language=None, model_name=None):

"""Transcribe audio file using OpenAI Whisper API"""

if not OPENAI_AVAILABLE or not openai_client:

raise RuntimeError(f"OpenAI API is not available: {OPENAI_ERROR}")

if not os.path.exists(file_path):

raise FileNotFoundError(f"File not found: {file_path}")

Check file size (OpenAI Whisper API max size is 25MB)

file_size = os.path.getsize(file_path)

max_size = 25 * 1024 * 1024 # 25MB

if file_size > max_size:

raise ValueError(f"File size {file_size} exceeds max size {max_size} ({file_size / 1024 / 1024:.2f}MB).")

OpenAI Whisper API

language (ISO 639-1) (e.g., 'ko', 'en')

model (e.g., 'whisper-1') (Default: 'whisper-1')

```
ai_services.py import WHISPER_LANGUAGE
```

```
try:
```

```
    # í ì ... ì .£,°
```

```
    with open(file_path, 'rb') as audio_file:
```

```
        # í ì ...°" ì ì í ì ¥ì ì ¶ì ¶ì í ì < í ì ...°" ì § ì
```

```
        filename = os.path.basename(file_path)
```

```
        transcript = openai_client.audio.transcriptions.create(
```

```
            model="whisper-1",
```

```
            file=(filename, audio_file, None), # (filename, file_object, content_type) í í í
```

```
            language=language or WHISPER_LANGUAGE or None, # Noneì .°'. ì ° £° ì §
```

```
            response_format="text"
```

```
        )
```

```
        return transcript
```

```
except Exception as e:
```

```
    error_msg = f"STT °‡ í ì / ì /°¥ °° ì : {str(e)}"
```

```
    # OpenAI API ì /°¥ °' ì ì § £° ì
```

```
    if "timeout" in str(e).lower():
```

```
        error_msg += "\n\ní ì ì ì ì . °° ì í ì µ° ° /. í ì ...ì . ° °<. í <£-°° ° /í ,ì
```

```
    elif "invalid" in str(e).lower() or "format" in str(e).lower():
```

```
        error_msg += "\n\nì § ì ° ì § ì ° í ì ... í ì ì ... ì ì ì µ° ° /. ì § ì í ì :
```

```
    elif "rate_limit" in str(e).lower():
```

```
        error_msg += "\n\nAPI ì <ì '° ì í ì ° ° <í ì µ° ° /. ì ì í ° /ì ì ° í
```

```
    raise RuntimeError(error_msg) from e
```

```
def generate_keywords(text):
```

```
    """í ì /í ,ì ì í /ì ° ì ¶ì ¶ì (OpenAI)"""
```

```
    if not OPENAI_AVAILABLE or not openai_client:
```

```
        raise RuntimeError(f"OpenAI API°¥... ì <ì 'í ì ì ì µ° ° /: {OPENAI_ERROR}")
```

```
    prompt = f""""° /ì í ì /í ,ì ì ì £...ì í /ì ° °¥... 5-10£° ì ¶ì ¶ì í ì £...ì ,ì .
```

```
    í /ì ° ° ì ...ì °ì £µ<°¶ í ì < í £µ>ì .°ì ° ì í ì £...ì ,ì .
```

```
    í ì /í ,:
```

```
{text}
```

```
    í /ì ° :"""
```

```
    from env_utils import OPENAI_MODEL
```

```
    response = openai_client.chat.completions.create(
```

```
        model=OPENAI_MODEL,
```

```
        messages=[{"role": "user", "content": prompt}]
```

```
    )
```

```
    return response.choices[0].message.content.strip()
```

```
def generate_summary(text):
```

```
    """í ì /í , ì ì %ì ì - (OpenAI)"""
```

```
    if not OPENAI_AVAILABLE or not openai_client:
```

```
        raise RuntimeError(f"OpenAI API°¥... ì <ì 'í ì ì ì µ° ° /: {OPENAI_ERROR}")
```

```
ai_services.py /ì í ì /í ,°¥... í μì < ° ì '°$ £° ° í ì ì %í ì £...ì ,ì (3-5°<,ì ¥).
```

```
í ì /í ,:
```

```
{text}
```

```
ì ì %:"""
```

```
from env_utils import OPENAI_MODEL
response = openai_client.chat.completions.create(
    model=OPENAI_MODEL,
    messages=[{"role": "user", "content": prompt}]
)
```

```
return response.choices[0].message.content.strip()
```

```
def generate_quiz(text, count=5):
```

```
    """í ì f ì ì - (OpenAI) - ì <° < °<,ì """
```

```
    if not OPENAI_AVAILABLE or not openai_client:
```

```
        raise RuntimeError(f"OpenAI API°¥... ì <ì 'í ì ì ì μ° ° /: {OPENAI_ERROR}")
```

```
    import json
```

```
    prompt = f"""° /ì í ì /í ,°¥... £,°° ì ...°ì {count}£° ì í ì f °<,ì °¥... ì ì -í ì £...ì ,ì .
    £° °<,ì ° ì £...£. ì í ì ì ì ì ì ... í °'°, ì ° μ£†... í ì /ì í <í í ì £...ì ,ì .
```

```
í ì /í ,:
```

```
{text}
```

```
JSON í ì ì ...°ì ì ° μí ì £...ì ,ì :
```

```
{{
```

```
    "questions": [
```

```
        {{
```

```
            "question": "°<,ì ° ì '",
```

```
            "answer": "ì ° μ",
```

```
            "explanation": "í ì /"
```

```
        }}
```

```
    ]
```

```
}}"""
```

```
from env_utils import OPENAI_MODEL
```

```
response = openai_client.chat.completions.create(
```

```
    model=OPENAI_MODEL,
```

```
    messages=[{"role": "user", "content": prompt}]
```

```
)
```

```
result_text = response.choices[0].message.content
```

```
# JSON ì ¶ ì ¶ (°$ í <° /ì ì ì %° °, °ì ì £-°)
```

```
if '''json' in result_text:
```

```
    result_text = result_text.split(''''json')[1].split('''')[0].strip()
```

```
elif ''' in result_text:
```

```
    result_text = result_text.split('''')[1].split('''')[0].strip()
```

```
return json.loads(result_text)
```

ai_services.py

```
def generate_single_quiz(text, existing_questions=None):
    """í .ì f í °<,ì ì ì - (OpenAI) - Æ,°ì i. °<,ì ì ì / °±μ° ì § ì ° °i """
    if not OPENAI_AVAILABLE or not openai_client:
        raise RuntimeError(f"OpenAI API°¥... ì <ì 'í ì ì ì μ° ° /: {OPENAI_ERROR}")

    import json

    existing_text = ""
    if existing_questions:
        existing_text = "\n\nì .°fl, ì ì -° °<,ì ° / (ì / °±μ° ì § ì ° °i ): \n"
        for idx, q in enumerate(existing_questions, 1):
            existing_text += f"{idx}. {q.get('question', '')} \n"

    prompt = f""""° /ì í ì /í ,°¥... Æ,°°° ì ...°i í .ì f °<,ì 1Æ° °¥... ì ì -í .ì £...ì ,ì .
ì £...Æ. ì í ì ì ì .ì .ì ... í °'°, ì ° μÆ±... í .ì /ì í <í ±í .ì £...ì ,ì .
{existing_text}

í ì /í ,:
{text}

JSON í ì ì ...°i ì ° μí .ì £...ì ,ì :
{{
    "question": "°<,ì ° .ì '",
    "answer": "ì ° μ",
    "explanation": "í .ì /"
}}""""

    from env_utils import OPENAI_MODEL
    response = openai_client.chat.completions.create(
        model=OPENAI_MODEL,
        messages=[{"role": "user", "content": prompt}]
    )

    result_text = response.choices[0].message.content
    # JSON ì ¶ ì ¶ (°§ í <° /ì .ì %° °, °i ì Æ-°)
    if ""'"json" in result_text:
        result_text = result_text.split(""'"json") [1].split(""'"") [0].strip()
    elif ""'" in result_text:
        result_text = result_text.split(""'"") [1].split(""'"") [0].strip()

    return json.loads(result_text)


def analyze_text(text, analysis_type="summary"):
    """í ì /í , °¶ ì (í /ì ° ° ° ì ì %) """
    if analysis_type == "keyword":
        return generate_keywords(text)
    else:
        return generate_summary(text)
```